

Validation and scoring

Simon Frost

Table of contents

1	Overview	1
2	Setup	2
3	A dataset of cases	2
4	Holding out by a group	3
5	Predicting a withheld target	4
6	Proper scoring rules	4
7	Calibration	5
8	Beating the prior	5
9	Ranking the variables by mutual information	7
10	Summary	8
11	References	8

1 Overview

A Bayesian network that looks plausible and answers queries coherently has not been shown to predict anything. Reviews of ecological Bayesian networks return to this point repeatedly ([Marcot et al. 2006](#); [Chen and Pollino 2012](#)): sensitivity analysis is not validation, and apparent fit is not forecast quality. What is asked for instead is out-of-sample, decision-focused validation — a holdout separated in space or time, a calibration assessment, proper scoring rules such as the Brier and log scores, and a comparison against a baseline that the network has to beat.

This vignette walks through that workflow on the SPEC section 45 reference habitat model: simulate a dataset of cases with a seeded generator, hold out whole groups of a grouping variable,

predict a withheld target, score the predictions, tabulate the calibration curve, compare against the marginal prior, and finally rank the variables by how much they would reduce the uncertainty of the target.

One honesty note before the numbers. The cases here are *simulated from the model that is then scored*, so this is a check of the machinery and a demonstration of the reporting, not evidence about any real system. In a real study the cases come from monitoring and the network's tables come from elsewhere; every number below would then mean what it says.

2 Setup

```
using BayesianNetworks
using BayesianNetworkInference
using Random
```

```
m = reference_habitat_model()
variables(compile(m))
```

```
7-element Vector{Symbol}:
 :Climate
 :Irrigation
 :SoilMoisture
 :GrazingPressure
 :Vegetation
 :HabitatQuality
 :Occupancy
```

3 A dataset of cases

A dataset is a `Cases`: a vector of `Dict{Symbol, Symbol}` observations, each mapping a variable to an observed state. Cases may be complete or partial — a variable a case does not record is simply not entered as evidence, which is the normal situation with ecological monitoring. `Cases(ancestral_sample(m, n; rng))` turns a seeded sample table into complete cases.

```
rng = MersenneTwister(20260907)
cases = Cases(ancestral_sample(m, 600; rng))
```

Cases with 600 observations over 7 variables

```
1: [:Climate => :normal, :GrazingPressure => :low, :HabitatQuality => :good,
 :Irrigation => :high, :Occupancy => :present, :SoilMoisture => :medium,
 :Vegetation => :moderate]
2: [:Climate => :normal, :GrazingPressure => :low, :HabitatQuality => :good,
 :Irrigation => :low, :Occupancy => :present, :SoilMoisture => :high, :Vegetation
 => :moderate]
```

```
3: [:Climate => :normal, :GrazingPressure => :low, :HabitatQuality => :poor,
: Irrigation => :high, :Occupancy => :present, :SoilMoisture => :medium,
:Vegetation => :moderate]
... (597 more)
```

```
variables(cases)
```

```
7-element Vector{Symbol}:
:Climate
:GrazingPressure
:HabitatQuality
:Irrigation
:Occupancy
:SoilMoisture
:Vegetation
```

```
cases[1]
```

```
Dict{Symbol, Symbol} with 7 entries:
:Vegetation      => :moderate
:GrazingPressure => :low
:HabitatQuality  => :good
:SoilMoisture    => :medium
:Occupancy       => :present
:Climate         => :normal
:Irrigation      => :high
```

4 Holding out by a group

`holdout(cases; by, fraction, rng)` returns two index vectors. Without `by` it draws cases uniformly; with `by = :X` it groups the cases by their observed state of `X` and draws whole groups, so no group is ever split between the two sides. That is how a spatial or temporal holdout is expressed here: record the catchment, region or year as a variable of the case and hold out by it. Because whole groups move together the requested fraction is only approximate: a shuffled group joins the test set only while that brings the split closer to the target, and with three climate states there is no way to hold out exactly 30% without cutting one of them.

```
train, test = holdout(cases; by=:Climate, fraction=0.3, rng)
(train=length(train), test=length(test))
```

```
(train = 434, test = 166)
```

```
climates = sort(unique(c[:Climate] for c in cases))
[(climate=c,
 in_train=count(i -> cases[i][:Climate] == c, train),
 in_test=count(i -> cases[i][:Climate] == c, test)) for c in climates]
```

```
3-element Vector{@NamedTuple{climate::Symbol, in_train::Int64, in_test::Int64}}:
 (climate = :dry, in_train = 0, in_test = 166)
 (climate = :normal, in_train = 331, in_test = 0)
 (climate = :wet, in_train = 103, in_test = 0)
```

Every climate state sits entirely on one side. `kfold(cases; k, by, rng)` does the same for `k` folds. Neither function refits anything: this package has no parameter estimation, so the honest use of a split here is to score tables that came from elsewhere on cases they did not see. Refitting per fold would mean re-estimating every conditional probability table from the training indices and rebuilding the model with `bind_cpt` before scoring each test fold.

5 Predicting a withheld target

`predict` withholds the target and enters the rest of the case (restricted to `evidence_vars`) as evidence. Here the surveyor is assumed to see the vegetation and the habitat quality but not the occupancy.

```
held = cases[test]
p = predict(m, held, :Occupancy; evidence_vars=[:Vegetation, :HabitatQuality])
```

Predictions of `:Occupancy` for 166 cases over 2 states

```
round.(p.probabilities[1:5, :]; digits=4)
```

```
5×2 Matrix{Float64}:
 0.25  0.75
 0.8   0.2
 0.25  0.75
 0.8   0.2
 0.25  0.75
```

6 Proper scoring rules

A proper scoring rule is one that a forecaster cannot improve by misreporting their beliefs. The multi-category Brier score is the mean squared distance between the predicted distribution and the indicator of what happened (lower is better); the log score is the mean log probability of what happened (higher is better; the `floor` keyword decides what a zero-probability outcome costs); the spherical score is a bounded alternative that stays finite when a realised outcome was given probability zero.

```
(brier=round(brier_score(p); digits=4),
 log=round(log_score(p); digits=4),
 spherical=round(spherical_score(p); digits=4))
```

```
(brier = 0.299, log = -0.4766, spherical = 0.8376)
```

Accuracy of the maximum-a-posteriori state and its confusion matrix are reported beside those scores, never instead of them: accuracy throws away how confident the network was.

```
confusion_matrix(p)
```

ConfusionMatrix over 2 states (accuracy 0.8193)

observed \ predicted	absent	present
absent	80	14
present	16	56

7 Calibration

Calibration asks whether the predicted probabilities correspond to observed frequencies: of the cases where the network said 0.7, about 70% should have been occupied. `calibration_curve` bins the predicted probability of one state and reports the mean prediction, the observed frequency and the count of each bin; `calibration_error` is the count-weighted mean gap, the expected calibration error.

```
curve = calibration_curve(p, :present; bins=5)
```

CalibrationCurve with 5 bins over 166 cases (ECE 0.0404)

centre	predicted	observed	count
0.1	0.2	0.1667	96
0.3	NaN	NaN	0
0.5	NaN	NaN	0
0.7	0.75	0.8	70
0.9	NaN	NaN	0

```
round(calibration_error(p, :present; bins=5); digits=4)
```

0.0404

The network emits only two distinct probabilities here, one for each habitat-quality state, so only two of the five bins are populated and the curve is a coarse instrument. With real, more varied evidence the populated bins are the ones to read, and the count column says how much each of them is worth.

8 Beating the prior

The comparison that matters is against a baseline the network has to beat. `baseline` is the “always predict the marginal prior” forecaster: it ignores the evidence completely.

```
prior = baseline(m, held, :Occupancy)
round.(prior.probabilities[1, :]; digits=4)
```

```
2-element Vector{Float64}:
 0.5238
 0.4762
```

`evaluate` computes every score for both and prints them side by side.

```
report = evaluate(m, held, :Occupancy; evidence_vars=[:Vegetation, :HabitatQuality])
```

```
EvaluationResult for :Occupancy on 166 cases
evidence: Vegetation, HabitatQuality
metric          network      prior
Brier score (lower better)    0.299    0.4948
log score (higher better)    -0.4766  -0.688
spherical score (higher)     0.8376    0.7108
accuracy (MAP)               0.8193    0.5663
calibration error (ECE)       0.0404    0.0425
AUC(:present)                0.8124    0.5
```

Discrimination — the quality of the *ranking* the network induces — is the area under the ROC curve for a chosen state, one versus rest. A prior baseline gives every case the same score, so its AUC is exactly 0.5. AUC is only worth reporting when ranking cases is a sensible thing to do with the target, for instance when prioritising sites for survey; a well-ranked but badly calibrated network is still a bad probability statement.

```
(auc_network=round(auc(p, :present); digits=4),
 auc_prior=round(auc(prior, :present); digits=4))
```

```
(auc_network = 0.8124, auc_prior = 0.5)
```

```
r = roc_curve(p, :present)
[(threshold=round(t; digits=4), fpr=round(f; digits=4), tpr=round(v; digits=4))
 for (t, f, v) in zip(r.thresholds, r.fpr, r.tpr)]
```

```
4-element Vector{@NamedTuple{threshold::Float64, fpr::Float64, tpr::Float64}}:
 (threshold = Inf, fpr = 0.0, tpr = 0.0)
 (threshold = 0.75, fpr = 0.0426, tpr = 0.1944)
 (threshold = 0.75, fpr = 0.1489, tpr = 0.7778)
 (threshold = 0.2, fpr = 1.0, tpr = 1.0)
```

The curve has very few points because, given the habitat quality, the vegetation tells the network nothing further about occupancy — the two are d-separated — so every case receives one of only two posteriors, up to floating-point differences in how each was computed. That is also why the calibration curve above populated only two of its bins, and it is exactly what the mutual-information ranking below makes explicit.

9 Ranking the variables by mutual information

Validation says whether the network predicts; sensitivity says what it depends on. sensitivity ranks every other variable by its mutual information with the target and reports that as a fraction of the target's entropy — the proportion of the remaining uncertainty that observing the variable would remove on average.

```
[(variable=r.variable,
  bits=round(r.mutual_information; digits=4),
  entropy_reduction=round(r.entropy_reduction; digits=4))
 for r in sensitivity(m, :Occupancy)]
```

```
6-element Vector{@NamedTuple{variable::Symbol, bits::Float64,
entropy_reduction::Float64}}:
 (variable = :HabitatQuality, bits = 0.2316, entropy_reduction = 0.2319)
 (variable = :Vegetation, bits = 0.0717, entropy_reduction = 0.0718)
 (variable = :SoilMoisture, bits = 0.0171, entropy_reduction = 0.0172)
 (variable = :GrazingPressure, bits = 0.0036, entropy_reduction = 0.0036)
 (variable = :Climate, bits = 0.0031, entropy_reduction = 0.0032)
 (variable = :Irrigation, bits = 0.0015, entropy_reduction = 0.0015)
```

```
round(entropy(m, :Occupancy); digits=4)
```

0.9984

Mutual information is exactly zero for variables the evidence d-separates from the target. Conditioning on the whole Markov blanket of Occupancy — here just HabitatQuality — makes every other variable uninformative:

```
[(variable=r.variable, bits=round(r.mutual_information; digits=12))
 for r in sensitivity(m, :Occupancy; evidence=Dict{:HabitatQuality => :good})]
```

```
5-element Vector{@NamedTuple{variable::Symbol, bits::Float64}}:
 (variable = :SoilMoisture, bits = 0.0)
 (variable = :Irrigation, bits = 0.0)
 (variable = :Vegetation, bits = 0.0)
 (variable = :GrazingPressure, bits = 0.0)
 (variable = :Climate, bits = 0.0)
```

Where the ranking averages over what a variable might turn out to be, tornado reports the extremes: how far a single finding could move the answer.

```
[(variable=r.variable, low=round(r.low; digits=4), high=round(r.high; digits=4),
  range=round(r.range; digits=4), low_state=r.low_state, high_state=r.high_state)
 for r in tornado(m, :Occupancy, :present)]
```

```

6-element Vector{@NamedTuple{variable::Symbol, low::Float64, high::Float64,
range::Float64, low_state::Symbol, high_state::Symbol}}:
 (variable = :HabitatQuality, low = 0.2, high = 0.75, range = 0.55, low_state =
:poor, high_state = :good)
 (variable = :Vegetation, low = 0.2825, high = 0.6675, range = 0.385, low_state =
:sparse, high_state = :dense)
 (variable = :SoilMoisture, low = 0.3671, high = 0.5644, range = 0.1973, low_state =
:low, high_state = :high)
 (variable = :Climate, low = 0.4306, high = 0.5222, range = 0.0915, low_state =
:dry, high_state = :wet)
 (variable = :GrazingPressure, low = 0.4408, high = 0.5117, range = 0.0708,
low_state = :high, high_state = :low)
 (variable = :Irrigation, low = 0.4579, high = 0.5037, range = 0.0458, low_state =
:low, high_state = :high)

```

10 Summary

- A dataset is a `Cases`: a vector of `Dict{Symbol, Symbol}` observations, complete or partial. `Cases(ancestral_sample(m, n; rng))` simulates one reproducibly.
- `holdout` and `kfold` produce index splits; by keeps whole groups together, which is how spatial and temporal holdout is expressed. They split cases and do not refit conditional probability tables.
- `predict` withholds the target and returns one posterior per case; `brier_score`, `log_score` and `spherical_score` are proper scoring rules over those posteriors, and `accuracy` with `confusion_matrix` describes the maximum-a-posteriori classification.
- `calibration_curve` and `calibration_error` check that stated probabilities match observed frequencies; `roc_curve` and `auc` describe discrimination, and only when a ranking interpretation is meaningful.
- `baseline` is the marginal prior, and `evaluate` prints the whole report against it, so “does the network beat always predicting the prior?” has an answer.
- `sensitivity`, `entropy`, `mutual_information` and `tornado` say what the model depends on. They describe the model, not the world; they are not a substitute for the scores above.

This is the last vignette of `BayesianNetworkInference.jl`. The scoring machinery is applied to a published ecological network in the “held-out validation” vignette of `EcologicalBayesianNetworks.jl`, and the same ideas carry over to decisions rather than predictions in `InfluenceDiagrams.jl`.

11 References

Brier ([1950](#)) introduced the score that carries his name, as a verification measure for probabilistic weather forecasts. Gneiting and Raftery ([2007](#)) is the modern reference for proper scoring rules, including the logarithmic and spherical scores used here, and for why propriety is the property that matters when a forecast will be acted on. Marcot ([2012](#)) sets out the performance

and uncertainty metrics expected of an ecological Bayesian network, including the entropy-reduction (mutual information) sensitivity metric that sensitivity computes. Chen and Pollino (2012) give the good-practice account of the whole life cycle, in which out-of-sample validation, calibration and a baseline comparison are quality gates rather than optional extras.

Brier, Glenn W. 1950. "Verification of Forecasts Expressed in Terms of Probability." *Monthly Weather Review* 78 (1): 1–3. [https://doi.org/10.1175/1520-0493\(1950\)078%3C0001:VOFEIT%3E2.0.CO;2](https://doi.org/10.1175/1520-0493(1950)078%3C0001:VOFEIT%3E2.0.CO;2).

Chen, Serena H., and Carmel A. Pollino. 2012. "Good Practice in Bayesian Network Modelling." *Environmental Modelling & Software* 37: 134–45. <https://doi.org/10.1016/j.envsoft.2012.03.016>.

Gneiting, Tilmann, and Adrian E. Raftery. 2007. "Strictly Proper Scoring Rules, Prediction, and Estimation." *Journal of the American Statistical Association* 102 (477): 359–78. <https://doi.org/10.1198/016214506000001437>.

Marcot, Bruce G. 2012. "Metrics for Evaluating Performance and Uncertainty of Bayesian Network Models." *Ecological Modelling* 230: 50–62. <https://doi.org/10.1016/j.ecolmodel.2012.01.013>.

Marcot, Bruce G., J. Douglas Steventon, Glenn D. Sutherland, and Robert K. McCann. 2006. "Guidelines for Developing and Updating Bayesian Belief Networks Applied to Ecological Modeling and Conservation." *Canadian Journal of Forest Research* 36 (12): 3063–74. <https://doi.org/10.1139/x06-135>.